

Python, FastAPI, PostgreSQL & ML/AI

2-Day Technical Assessment

Candidate: GOBI KRISHNA P

Submission Deadline: 22/09/2026 before 12:30 PM

****IMPORTANT - CANDIDATE REQUIREMENT**

This assessment must be completed independently by the candidate.

The candidate must NOT use AI tools to generate, write, modify, debug, or complete the assessment, including but not limited to:

- ChatGPT
- Claude
- Gemini
- GitHub Copilot
- Cursor AI
- Windsurf
- Antigravity AI
- other AI coding assistants or code-generation tools

The purpose of this assessment is to evaluate the candidate's own understanding of Python programming, FastAPI, REST API development, PostgreSQL, SQL and database design, indexing, architecture, data cleaning and validation, feature engineering, machine learning, and AI API integration.

The candidate may use official technical documentation for Python, FastAPI, PostgreSQL, SQLAlchemy, scikit-learn, OpenRouter, and other libraries.

The candidate must be able to explain and defend all implementation decisions and code during the technical interview. If the candidate cannot explain a significant portion of the submitted implementation, the assessment may be considered invalid.

Neon website: <https://neon.com/>

1. Objective

- Develop APIs using Python and FastAPI.
- Connect FastAPI to Neon PostgreSQL.
- Design relational database tables and appropriate indexes.
- Implement CRUD operations and database-level filtering.
- Implement SQL INNER JOIN and LEFT JOIN.
- Perform data cleaning and validation.
- Generate and process at least 5,000 records.
- Perform feature engineering.
- Train and evaluate an ML model.
- Generate a structured ML output dataset consumable by AI.
- Integrate an AI model using OpenRouter.
- Use ML-generated data to produce AI-generated insights/recommendations.

- Organize the project using clean, maintainable architecture.

2. Technology Requirements

Area	Required Technology
Backend	Python, FastAPI, Pydantic, SQLAlchemy or another reasonable PostgreSQL database library
Database	PostgreSQL using Neon PostgreSQL
Machine Learning	Pandas, NumPy, Scikit-learn; visualization libraries optional
AI	OpenRouter API; preferred model: NVIDIA Nemotron Ultra 3

If NVIDIA Nemotron Ultra 3 is unavailable, unsupported, or does not work through the candidate's OpenRouter account, another available OpenRouter model may be used. Document the model used and, if applicable, why a fallback was necessary.

3. Suggested Project Scenario

Build a Customer Order & Business Intelligence API. The system should maintain customer and order information, provide APIs for managing the data, train an ML model using generated historical data, and use the ML results to generate business insights/recommendations through an AI model.

PART A — FASTAPI + POSTGRESQL

4. Database Setup

Create a PostgreSQL database using Neon. The application must connect to the Neon database using environment variables.

```
DATABASE_URL=postgresql://...
```

```
OPENROUTER_API_KEY=...
```

- Do not hardcode database credentials.
- Do not hardcode the OpenRouter API key.
- Use .env locally.
- Provide a .env.example file.
- Do not commit secrets to Git.

5. Database Tables

Customers

```
customers
```

```
-----
```

```
id
name
email
phone
city
created_at
updated_at
```

- id should be the primary key.
- email should have an appropriate uniqueness constraint.

- Use appropriate data types.
- Add timestamps.

Orders

orders

id
customer_id
order_date
product_name
category
quantity
unit_price
total_amount
status
created_at
updated_at

- id should be the primary key.
- customer_id should reference customers.id.
- Use an appropriate foreign key.
- Use appropriate numeric types for monetary values.
- Define a reasonable set of order statuses.

pending | confirmed | shipped | delivered | cancelled

The candidate may add additional fields if useful.

6. Database Indexing

Create appropriate indexes supporting customer lookup/filtering, order lookup by customer_id, order filtering by status, and order date-based queries.

Document which indexes were created, why they were created, and which queries benefit from them. Do not blindly create indexes on every column.

```
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
```

7. Database Migration / Table Creation

Provide a reproducible way to create the database schema. Acceptable approaches include SQL migration files, Alembic, SQLAlchemy metadata/table creation, or another structured migration approach.

PART B — FASTAPI APIs

8. Customer CRUD APIs

Operation	Endpoint	Requirement
Create	POST /customers	Validate request data and return created customer.
Get	GET /customers/{customer_id}	Return customer or appropriate not-found error.
List	GET /customers	Return customers with reasonable pagination.
Update	PUT /customers/{customer_id}	Update existing customer.
Delete	DELETE /customers/{customer_id}	Delete customer or use a clearly documented deletion strategy.

GET /customers?page=1&limit=20

9. Customer Filtering API

Implement filtering functionality. Example:

```
GET /customers?name=john
```

Filtering must be implemented at the database/query level, not by retrieving the entire customer table and filtering it in Python. Additional filters such as email or city are optional.

10. Order APIs

Implement APIs to create, retrieve, retrieve orders for a customer, update, and delete orders.

```
GET /customers/{customer_id}/orders
```

Support at least one useful filter, such as status, date range, or category.

11. INNER JOIN Requirement

At least one API must use an actual SQL/ORM INNER JOIN. Suggested endpoint:

```
GET /reports/customer-orders
```

```
[
  {
    "customer_id": 1,
    "customer_name": "John",
    "order_id": 1001,
    "product_name": "Laptop",
    "total_amount": 75000
  }
]
```

The candidate must explain why INNER JOIN was used.

12. LEFT JOIN Requirement

At least one API must use an actual SQL/ORM LEFT JOIN. Suggested endpoint:

```
GET /reports/customer-summary
```

```
[
  {
    "customer_id": 1,
    "customer_name": "John",
    "total_orders": 5,
    "total_amount": 125000
  },
  {
    "customer_id": 2,
    "customer_name": "Sarah",
    "total_orders": 0,
    "total_amount": 0
  }
]
```

The response must include customers even if they have no orders. The candidate must explain why LEFT JOIN is required instead of INNER JOIN.

13. API Validation & Error Handling

Handle invalid customer IDs, non-existent customers, invalid emails, invalid quantities/prices, invalid order status, missing required fields, duplicate emails, invalid parameters, and database errors.

Use appropriate HTTP status codes such as 200, 201, 400, 404, 409, 422, and 500. Avoid exposing raw database errors to API consumers.

PART C — DATA GENERATION & DATA PROCESSING

14. Generate Dataset

Create a synthetic dataset containing at least 5,000 records. The candidate may generate the data using Python.

Suggested domain: Customer purchase/order prediction.

```
customer_id
age
city
customer_segment
product_category
order_count
average_order_value
total_spend
discount_percentage
days_since_last_order
website_visits
support_tickets
return_count
payment_type
customer_tenure_days
```

The candidate may create additional fields.

15. Data Quality Problems

The generated dataset should contain realistic data-quality issues, such as:

- missing values
- duplicate records
- inconsistent categorical values
- invalid numeric values
- outliers
- incorrect data types
- inconsistent text formatting

The candidate must identify and handle these problems.

16. Data Cleaning

Missing-value handling

- numeric → median or another justified strategy
- categorical → mode/Unknown or another justified strategy
- explain the choice

Duplicate handling

- identify and remove duplicate records where appropriate

Data-type validation

- numeric columns → numeric
- dates → datetime
- categorical fields → consistent values

Categorical normalization

- normalize values such as "Chennai", "chennai", and " CHENNAI "

PART D — FEATURE ENGINEERING**17. Feature Engineering**

Create at least 5 meaningful engineered features. Examples:

```
average_order_value
customer_lifetime_value
purchase_frequency
return_rate
engagement_score
recency_score
```

For each feature, document the original feature(s), new feature, calculation, and why it may be useful.

```
return_rate = return_count / order_count
```

Handle division-by-zero cases appropriately.

18. Avoid Data Leakage

Explain what data was available at prediction time, what data was used for training, and what data must not be used. Identify any potential leakage risks.

PART E — MACHINE LEARNING**19. Define an ML Problem**

Define one ML problem based on the generated dataset. Examples:

- Customer churn prediction — churn = 0/1
- Customer spending prediction — predict future_spend
- Customer purchase probability — predict purchase_probability

Another reasonable business problem is acceptable.

20. Train ML Model

Train at least one machine-learning model using Scikit-learn or another appropriate ML library. Explain model selection, training data, target variable, input features, train/test split, preprocessing, and evaluation.

Possible models include Logistic Regression, Random Forest, Gradient Boosting, XGBoost, Linear Regression, or Random Forest Regressor.

21. Model Evaluation

Problem Type	Possible Metrics
Classification	Accuracy, Precision, Recall, F1, ROC-AUC
Regression	MAE, MSE, RMSE, R ²

Explain why the selected metrics are appropriate.

22. Generate ML Output Dataset

The ML model must generate a structured output dataset that can be consumed by an AI system.

```
customer_id
churn_probability
risk_level
top_risk_factors
customer_segment
recommended_action
prediction_date
model_version

{
  "customer_id": 1025,
  "churn_probability": 0.82,
  "risk_level": "high",
  "top_risk_factors": ["low engagement", "high return rate"],
  "recommended_action": "Customer retention campaign",
  "model_version": "v1.0"
}
```

The candidate must generate enough structured output to make the AI stage meaningful.

PART F — AI INTEGRATION

23. OpenRouter Integration

Integrate the application with the OpenRouter API.

Preferred model: NVIDIA Nemotron Ultra 3

If unavailable or unsupported, another available OpenRouter model may be used. Document the model used and the reason for fallback, if applicable.

24. AI Insight API

POST /ai/insights

The API should retrieve or receive ML-generated results, prepare structured context, send it to OpenRouter, request useful business insights/recommendations, and return the AI-generated result.

```
Request:
{
  "customer_id": 1025
}

Response:
{
  "customer_id": 1025,
  "risk_level": "high",
  "insight": "...",
  "recommendations": ["...", "..."]
}
```

25. AI Prompt Design

Create a structured prompt containing customer information, ML prediction, relevant features, risk indicators, and business context.

- A short summary.
- Key insights and factors influencing the prediction.
- Practical recommendations.
- Confidence/limitations, without inventing information not provided by the ML/data pipeline.

26. AI Output Validation

Ensure the AI response follows a predictable structure and is safely parsed/validated before being returned.

```
{
  "summary": "...",
  "key_insights": ["...", "..."],
  "recommendations": ["...", "..."],
  "limitations": ["..."]
}
```

PART G — ARCHITECTURE

27. Project Architecture

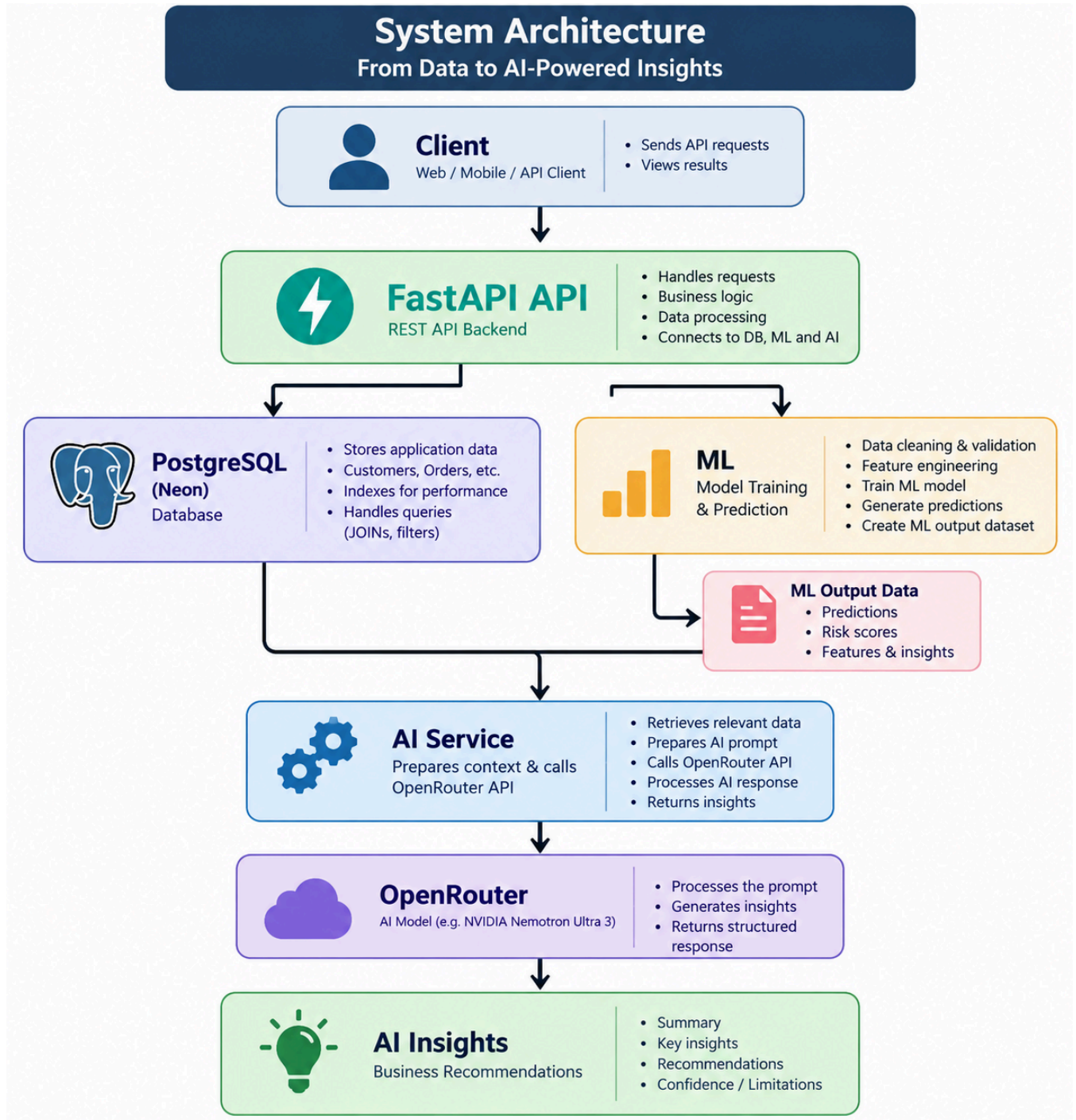
Organize the project using a maintainable structure. The following is an example; another reasonable structure is acceptable.

```
project/
├── app/
│   ├── main.py
│   ├── api/routes/
│   ├── schemas/
│   ├── models/
│   ├── services/
│   ├── repositories/
│   ├── database/
│   ├── ml/
│   ├── ai/
│   └── core/
├── scripts/generate_dataset.py
├── tests/
├── data/
├── .env.example
├── requirements.txt
├── README.md
└── ...
```

The important requirement is separation of responsibilities.

28. Architecture Explanation

Include a simple architecture diagram in the README and explain the API, database, service, ML, and AI layers, plus error handling and configuration/secrets.



PART H — TESTING

29. Testing Requirements

Implement tests for important functionality. At minimum, include tests for:

- Create customer
- Get customer
- Customer not found
- Customer filtering

- Create order
- Customer orders
- INNER JOIN API
- LEFT JOIN API
- dataset generation
- feature engineering
- model training/evaluation

100% coverage is not required; demonstrate that you understand how to test the application.

PART I — DOCUMENTATION

30. README Requirements

- Project Overview — what was built.
- Technology Stack.
- Setup Instructions — Python version, virtual environment, dependencies, environment variables, database setup, migration/setup, and startup.
- Environment Variables — provide .env.example without real credentials.
- API Documentation — method, URL, request, response, and error cases.
- Database Design — tables, relationships, and indexes.
- SQL JOIN Explanation — where and why INNER JOIN and LEFT JOIN were used.
- ML Approach — dataset, cleaning, validation, features, target, model, and metrics.
- AI Integration — OpenRouter integration, model used, prompt structure, response format, and error handling.
- Architecture — architecture diagram and explanation.

PART J — BONUS

Bonus 1 — Pagination

Implement robust pagination with page, limit, total, and total_pages.

Bonus 2 — Sorting

Example: GET /orders?sort_by=total_amount&order=desc

Bonus 3 — API Authentication

Implement API authentication using API key, JWT, or OAuth2.

Bonus 4 — Docker

Provide a Dockerfile for the FastAPI application.

Bonus 5 — Logging

Implement structured application logging.

Bonus 6 — Model Versioning

Include a model version in ML predictions, e.g. model_version = "v1.0".

Deliverables

1. Git repository / source code
2. README.md
3. requirements.txt
4. .env.example

5. Database migration/schema files
6. Dataset generation script
7. Generated dataset or generation instructions
8. ML training code
9. Trained model or reproducible training pipeline
10. FastAPI application
11. API tests
12. Architecture diagram
13. ML output dataset
14. AI integration
15. Sample API requests/responses

What Will Be Evaluated

Python

- clean Python code
- functions/classes where appropriate
- exception handling
- type hints
- reusable code
- understanding of Python concepts

FastAPI

- routing
- request/response models
- validation
- dependency injection
- error handling
- API structure

Database

- relational design
- primary/foreign keys
- SQL
- joins
- indexes
- transactions where appropriate
- query efficiency

Architecture

- separation of concerns
- maintainability
- scalability considerations
- configuration management
- service/repository separation

ML

- correct data preparation
- meaningful features
- correct train/test methodology
- appropriate evaluation metrics

- avoidance of data leakage

AI

- correct API integration
- secure API-key handling
- structured prompts
- structured output
- handling AI failures

After submission, the candidate may be asked to explain and demonstrate the implementation. The candidate should be able to demonstrate that they personally understand the submitted implementation.

Submission Deadline

22/09/2026 before 12:30 PM

The candidate has 2 calendar days from the time the assessment is provided to complete and submit the assignment.

The candidate must not use AI tools to complete the assessment.